

Agent acceleration

Making LLM agents that operate software finish a task in less time and for less money, without finishing fewer tasks.

Computer-use, web and coding agents, which work a desktop, a browser or a code base one step at a time through a model.

September 2026

PART 1

Where the time and the money go

Why LLM agents that operate software are slow and expensive, and how the two are linked.

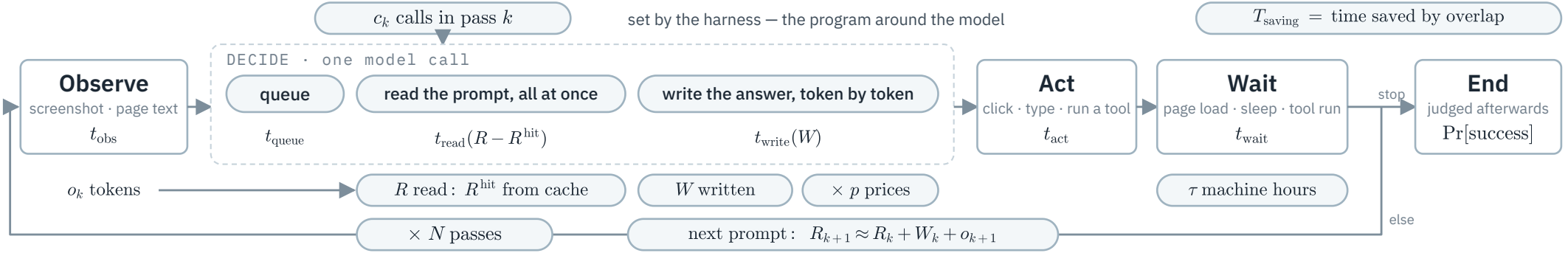
Every number comes from a published measurement of a computer-use, web or coding agent, or from a vendor’s own price page (2024–2026); the conditions of each measurement are stated next to it, and the full references are at the end of the part.

IN THIS PART

02	The agent loop and the variables at each step
03–06	Where the time goes: five sources, and which one dominates
07–08	Where the money goes: six sources, and four accounting conventions
09–10	How time and money are linked, and the levers
11–12	References · Appendix A0–A6

The agent loop and the variables at each step

here: the steps of one pass, the variable at each step, and how the variables add up to time and cost → to: the five sources of slowness



TIME
ELAPSED
p. 3–6

$$T_{\text{task}} = \sum_{k=1}^N \left[\sum_{j=1}^{c_k} \ell_{kj} + t_{\text{obs}} + t_{\text{act}} + t_{\text{wait}} \right] - T_{\text{saving}}$$
$$\ell_{kj} = t_{\text{queue}} + t_{\text{read}}(R_{kj} - R_{kj}^{\text{hit}}) + t_{\text{write}}(W_{kj}), \quad T_{\text{saving}} \geq 0$$

PROMPT
GROWS
p. 5, 7

$$R_{k+1} \approx R_k + W_k + o_{k+1} \Rightarrow \sum_{k=1}^N R_k \approx NR_1 + \frac{N(N-1)}{2} (\bar{w} + \bar{o})$$

MONEY
PER ATTEMPT
p. 7–8

$$\text{Cost}_{\text{task}} = \sum_{k,j} (R^{\text{hit}} p_{\text{hit}} + R^{\text{miss}} p_{\text{miss}} + R^{\text{store}} p_{\text{store}} + W p_{\text{write}}) + \tau p_{\text{env}}$$

SUCCESS
AND GOAL
p. 7–10

$$\text{Cost}_{\text{success}} = \frac{\mathbb{E}[\text{Cost}_{\text{task}}]}{\text{Pr}[\text{success}]} \quad \min (\mathbb{E}[T_{\text{task}}], \text{Cost}_{\text{success}}) \quad \text{s. t.} \quad \text{Pr}[\text{success}] \geq s_0$$

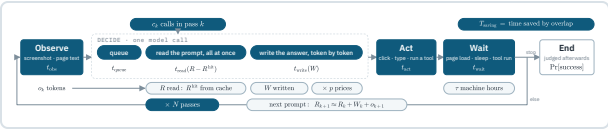
Time of one attempt T_{task} : pass k runs from 1 to N , call j from 1 to c_k . ℓ_{kj} is one call: queueing, reading the prompt tokens not in the cache ($R - R^{\text{hit}}$, prefill) and writing its W tokens (decode). $t_{\text{obs}}, t_{\text{act}}, t_{\text{wait}}$: observe, act, wait. T_{saving} : time saved by running steps at the same time; 0 if all serial. serial form: Kim et al., 2024 · overlap: Feng et al., 2026 · per-call split: Zhu et al., 2026

R_k : prompt tokens of pass k (R_1 : system prompt and task). Each prompt adds the answer W_k and the next observation o_{k+1} ; $\bar{w}, \bar{o}$: their means per pass. So reading grows with N^2 (calc.); measured on 39 OSWorld tasks: cost “grows quadratically with the number of steps”. recurrence: Yuan et al., 2026 · measured, GTA1 harness: Abhyankar, Qi & Zhang, 2026

Prompt tokens by billing class: R^{hit} read from the cache, R^{miss} not in it, R^{store} stored into it; W written tokens, thinking included; each p is the price per token of its class. τ : hours the environment machine is billed ($\tau \geq T_{\text{task}}$); p_{env} : its hourly price. Anthropic, 2026b; OpenAI, 2026 · cached vs new: Zhu et al., 2026 · machine: page 8

$\mathbb{E}[\cdot]$: mean over attempts; $\text{Pr}[\text{success}]$: chance that one attempt succeeds. Failed attempts are billed, so cost per success = mean cost per attempt ÷ success rate. Goal: lower time or cost per success without raising the other, keeping the success rate at least s_0 , its level before the change. cost per success: Erol et al., 2026 (per task; the benchmark average is ours) · goal: ours

Slowness has five sources



from the time equation → here: every cause of slowness, in five classes → to: the numbers behind each class

Four of the five are terms of the time equation; the fifth is how they combine: the overlap term T_{saving} is close to zero, so the terms add. Which class dominates depends on the harness, not on the kind of agent (page 6). Cause-by-cause tables with measurement conditions: Appendix A1.

$$T_{\text{task}} = \left| \sum_k \right| \quad \left| \sum_{j \leq c_k} \right| \quad \left| (t_{\text{queue}} + t_{\text{read}} + t_{\text{write}}) \right| \quad \left| + t_{\text{obs}} + t_{\text{act}} + t_{\text{wait}} \right| \quad \left| - T_{\text{saving}}, \quad T_{\text{saving}} \approx 0 \right|$$

I · PASSESII · CALLS PER PASSIII · EACH CALLIV · ENVIRONMENTV · SERIAL: LITTLE OVERLAPS

N passes per task · c_k model calls in pass k · $t_{\text{queue}}, t_{\text{read}}, t_{\text{write}}$ one call: queueing, reading the prompt (prefill), writing the answer (decode) · $t_{\text{obs}}, t_{\text{act}}, t_{\text{wait}}$ observe, act, wait · T_{saving} time saved by running steps at the same time

I · TOO MANY PASSES (N)
Hour-scale tasks take hundreds of passes, and every pass is a full round trip

- 318 tool calls per task on OSWorld 2.0 — 108 desktop tasks, Claude Opus 4.7, one action per step

(XLANG Lab, 2026)

II · TOO MANY CALLS PER PASS (c_k)
A harness that plans, judges and reflects multiplies every pass

- 4–12 planning calls per judging call — GTA1 harness: four parallel planners, up to three retries, one judge

(Abhyankar, Qi & Zhang, 2026)

III · EACH CALL IS SLOW
Inside a call, waiting, reading and writing are three separate delays

- Writing, one token at a time, is 91–98.6% of model time — local 27–31B models with a warm cache

(Yuan et al., 2026)

IV · THE ENVIRONMENT IS SLOW
The environment costs time whether or not the model is running

- 6.6 s per browser action vs 4.7 s per model call, median — 151 WebVoyager live-site tasks, GPT-4o

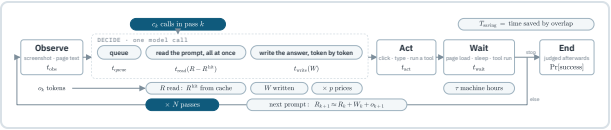
(Wong et al., 2026)

V · EVERYTHING IS SERIAL
Almost nothing overlaps, so task time is close to a sum: fixing one term saves only its own share

- Concurrency within a turn: 1.15 — GitHub Copilot production telemetry

(Liu et al., 2026)

I–II · Too many passes, too many calls per pass



from the five classes → here: the two multipliers, passes and calls per pass → to: what happens inside a call and around it

N multiplies every term and c_k multiplies N . Passes accumulate because tasks are long, each pass does one action, navigation-only passes still go through the model, and the agent idles or loops; calls accumulate because the harness plans, judges and reflects, or samples several candidates per step.

I-1, I-2 · LONG TASKS, ONE ACTION PER PASS

Long tasks with one action per pass need hundreds of passes; several actions per call cut them by half or more

- 318 tool calls per task with one action per step, 160.7 with several actions per call — OSWorld 2.0, 108 long tasks, Claude Opus 4.7

(XLANG Lab, 2026)

I-3 · NAVIGATION-ONLY PASSES

Most passes are navigation that needs no thinking, yet each one is a model call

- 66.7% of the steps in the median task are pure navigation — 151 WebVoyager tasks on live websites, three text agents driven by GPT-4o

(Wong et al., 2026)

I-4 · IDLING AND DEAD LOOPS

When an agent is stuck, every repeated pass is billed and timed in full

- One element-locating loop repeated a step 18 times: 27 minutes and \$8.47 at list price without caching — GTA1 harness, OSWorld

(Abhyankar, Qi & Zhang, 2026)

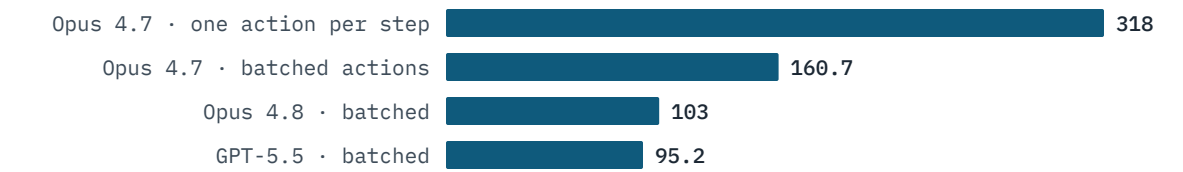
II · PLANNING, JUDGING, SAMPLING

Extra calls per pass buy accuracy at a rising token cost per point

- 10 candidates per step instead of 1: 38.8% → 43.2% success for 96K → 920K tokens per task — gpt-oss-120b, 165 WebArena-Lite tasks

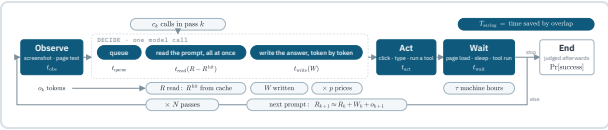
(Lee et al., 2026)

Steps per task on OSWorld 2.0, 108 tasks



XLANG Lab, 2026 · mean over tasks

III–V · Inside a call, around it, and in sequence



from the multipliers → here: the per-call and per-pass terms → to: which term dominates

Inside a call: queueing, reading, writing. Around it: observation, execution, fixed sleeps, tool runs. All of it runs strictly in sequence, so task time is the sum, not the largest term.

III-1, III-2 · QUEUEING AND READING

Before a single token is written, the call has waited in a queue and re-read the whole history

- Identical requests take up to 69× longer depending on the time of day — 15 models, 5 providers

(Bian et al., 2025)

III-3, III-4 · WRITING AND THINKING

Writing is the slowest segment per token, and thinking modes write far more tokens without reliably buying accuracy

- 224K vs 37K output tokens per task — Claude Opus 4.8 vs GPT-5.5 on the same 108 OSWorld 2.0 tasks

(XLANG Lab, 2026)

IV · THE ENVIRONMENT

The environment is slow on its own, and the benchmarks add fixed sleeps on top

- OSWorld 2.0 sleeps 3 s after every action: × 318 steps ≈ 16 min of pure waiting per task (calc.)

(XLANG Lab, 2026; xlang-ai, 2026)

V · SERIAL

Nothing overlaps, so task time is a sum — and a “total time” often includes the human

- 80–92% of a coding session’s elapsed time is the person thinking between turns — GitHub Copilot telemetry (median) and 4,265 Claude Code / Codex sessions

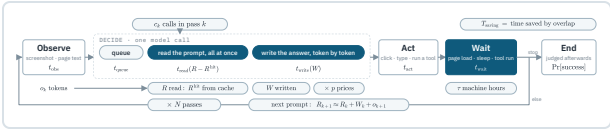
(Liu et al., 2026; Zhu et al., 2026)

Where a Claude Code / Codex request’s 4.3 minutes go



Zhu et al., 2026 · 4,265 sessions, 43 developers

Which term dominates depends on the harness, not on the kind of agent



from the five classes → here: the shares, per kind of agent → to: the six sources of cost

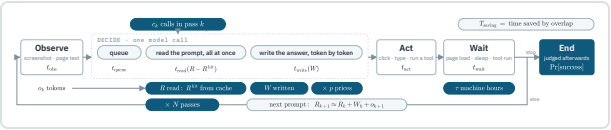
All five classes exist in every agent; the heaviest term differs, and it flips when the harness changes. The one multiplier common to all three kinds is the number of passes, N .

	SCREENSHOT DESKTOP AGENT, MULTI-CALL HARNESS	TEXT WEB AGENT	CODING AGENT WITH A PROMPT CACHE
HEAVIEST	Reading × calls per pass × passes. 87–97% of task time is planning, judging and reflection calls (GTA1, Agent S2; 39 OSWorld tasks) (Abhyankar, Qi & Zhang, 2026)	Browser execution and waiting. Per step, median: browser 6.6 s, model 4.7 s (151 WebVoyager live-site tasks, GPT-4o) (Wong et al., 2026)	Writing, or tool tails. Writing is 91–98.6% of model time (Yuan et al., 2026); in Claude Code / Codex requests tools take 59.8%, the model 41.0% (Zhu et al., 2026)
FLIPS WHEN	The harness makes one call per pass: over 70% of the time is then the sandbox, because the benchmark sleeps 2–3 s after every action (Chang et al., 2026; xlang-ai, 2026; XLANG Lab, 2026)	The observation grows: 4.8× the input moved the model’s share of step time from 46.9% to 61.6% (Chang et al., 2026)	Load evicts the cache: latency up to 7.14× (Kang et al., 2026); or the tools are light and the model dominates again (Copilot)
IN ONE LINE	Slow in reading: every pass re-reads the growing screenshot history	Slow in the environment — until the observation grows, then slow in reading	Reading is cached away; slow in writing and in the tools

1 · The harness sets the dominant term — calls per pass, observation size, fixed sleeps, cache on or off. The explanation “agents are slow because inference is slow” is accurate only for multi-call screenshot agents.

2 · N multiplies every term — one pass fewer saves a whole pass of time and money in all three kinds; OSWorld 2.0’s authors list “fewer environment rounds” as a goal in its own right.

Cost has six sources



from where the time goes → here: every cause of cost, in six classes → to: four accounting conventions, and the environment machine

Money comes from two things only — tokens read and tokens written, each times a price. Writing costs 5× reading; a cached read costs a small fraction of a normal read (DeepSeek-V4.1-Flash: \$0.006 vs \$0.30 per million tokens; other vendors in Appendix A3); fast mode costs 2× (Anthropic, 2026b; OpenAI, 2026; DeepSeek, 2026).

Cost_{success} ≈

$$\sum_k \sum_{j \leq c_k}$$

3 · NUMBER OF CALLS

$$(R^{\text{hit}}, R^{\text{miss}}, R^{\text{store}}$$

1 · TOKENS READ

$$, W)$$

2 · TOKENS WRITTEN

$$\cdot p$$

5 · PRICE PER CLASS

$$+ \tau p_{\text{env}}$$

6 · ENVIRONMENT

$$\div \text{Pr}[\text{success}]$$

4 · FAILURES, RETRIES

k, j pass, call · $R^{\text{hit}}, R^{\text{miss}}, R^{\text{store}}$ prompt tokens from the cache, not in it, stored into it · W written tokens · p price per token of each class · τp_{env} machine hours × hourly price · $\text{Pr}[\text{success}]$ success rate

1 · TOKENS READ

Reading is the larger bill, it grows with the square of the passes, and caching does not remove it

- Even at a 95.7% cache hit rate, prefix tokens are still 59.5% of the bill — 4,265 Claude Code / Codex sessions

(Zhu et al., 2026)

2 · TOKENS WRITTEN

Writing is the dearer token and thinking is billed as writing — but it is not the larger bill

- 31% of an uncached screenshot agent’s bill: \$2.43 counting output only vs \$7.87 counting all tokens — GTA1, 39 OSWorld tasks (calc.)

(Abhyankar, Qi & Zhang, 2026)

3 · NUMBER OF CALLS

The calls that made the agent slow are the same calls that make it expensive

- 5–13× the calls of a single-call harness per step — GTA1: planners, retries and a judge (calc.)

(Abhyankar, Qi & Zhang, 2026)

4 · FAILURES AND RETRIES

What a task costs is what a success costs, and failed attempts are billed in full

- \$72.4 per attempt ÷ 20.6% completion ≈ \$351 per success — OSWorld 2.0, best agent (calc.)

(XLANG Lab, 2026)

5 · UNIT PRICE

The price list sets the constant; speed tiers and long contexts double it

- Claude Opus 5.5 \$4 / \$20 vs GPT-6 Luna \$0.10 / \$0.50 per million tokens read / written — a 40× spread between two vendors’ list prices (calc.)

(Anthropic, 2026b; OpenAI, 2026)

6 · THE ENVIRONMENT MACHINE

Published task costs are API bills only; a CPU environment adds a small amount on top

- A one-hour CPU environment costs \$0.02–0.36 at list prices: 0.3–4.6% of a \$7.87 bill (calc.; next page)

vendor price pages, read 2026-09-27

SOURCES · one representative number per class; the complete tables with conditions are in Appendix A3

Appendix A3 →

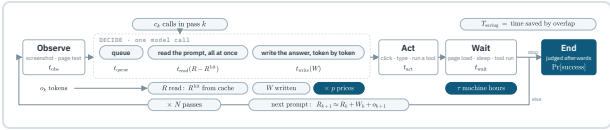
Four accounting conventions for one task, and the cost of the environment machine

from the six classes → here: which convention a cost figure uses, and what the environment adds → to: how slow and expensive are linked

The cost of one task depends on the accounting convention: the four conventions in use differ by up to 30×, and none includes the environment machine. A cost figure is comparable only with its convention stated.

CONVENTION	THE SAME RUN, COUNTED BOTH WAYS	GAP
OUTPUT ONLY VS ALL TOKENS	GTA1 on 39 OSWorld tasks, o3 list price, no cache: \$2.43 counting output only; \$7.87 counting all tokens (Abhyankar, Qi & Zhang, 2026)	3.2×
PER ATTEMPT VS PER SUCCESS	OSWorld 2.0, Claude Opus 4.8: \$72.4 per attempt at 20.6% completion → ≈ \$351 (calc.; a benchmark average, not the cost of retrying one given task) (XLANG Lab, 2026)	4.9×
UNCACHED VS CACHED	A cached read costs a fraction of a normal read (DeepSeek-V4.1-Flash: \$0.006 vs \$0.30 per million tokens; other vendors in A3); but at a 95.7% hit rate, prefix tokens were still 59.5% of the bill (4,265 Claude Code / Codex sessions) (DeepSeek, 2026; Zhu et al., 2026)	4–50× on the read price only, by vendor (calc.)
STANDARD VS FAST MODE	Claude Opus 5.5 \$4 / \$20 → \$8 / \$40; OpenAI 2× on every listed model; up to 2.5× faster writing (vendor-stated), reading unchanged (Anthropic, 2026b; OpenAI, 2026)	2×

The three that apply to a whole bill compound to about 31× (3.2 × 4.9 × 2, calc.). Screenshot-agent measurements do not report their cache state.



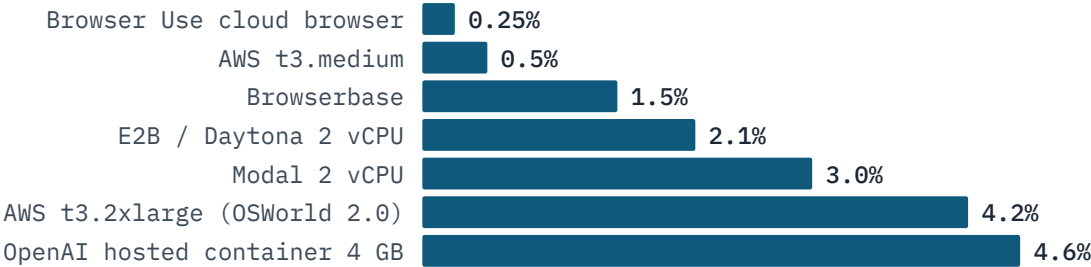
6 · THE ENVIRONMENT MACHINE, FROM LIST PRICES

For CPU environments the machine is a small addition; idle time, proxy traffic and GPUs change that

- An H100 sandbox at \$3.95 per hour is already half of a \$7.87 bill, and hourly billing runs while the agent waits — the median AgentSysBench session is active only 20% of its lifetime

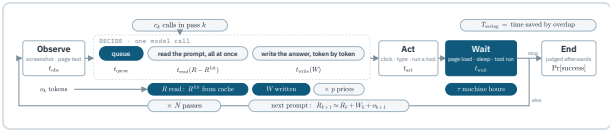
vendor price pages, read 2026-09-27; Chang et al., 2026

A one-hour CPU environment as a share of a \$7.87 API bill (calc.)



vendor price pages, read 2026-09-27 · full price table in Appendix A3

How slow and expensive are linked: three kinds of relationship



from the two bills → here: what each cause does to time and to money → to: the levers

The two equations share the read and write tokens; environment time and queueing reach the money equation only through the environment machine’s hours. So every cause relates to time and money in exactly one of three ways.

TIME

$$\sum_k \sum_j (t_{\text{queue}} + t_{\text{read}}(R - R^{\text{hit}}) + t_{\text{write}}(W)) + t_{\text{obs}} + t_{\text{act}} + t_{\text{wait}} - T_{\text{saving}}$$

MONEY

$$\sum_k \sum_j (R^{\text{hit}} p_{\text{hit}} + R^{\text{miss}} p_{\text{miss}} + R^{\text{store}} p_{\text{store}} + W p_{\text{write}}) + \tau p_{\text{env}} \div \text{Pr}[\text{success}]$$

in both equations — same source

no tokens, money only as machine hours — slow but not expensive

the price — money for time or accuracy

Σ_k, Σ_j over passes, over calls · $t_{\text{queue}}, t_{\text{read}}, t_{\text{write}}$ one call: queueing, reading the prompt (prefill), writing the answer (decode) · $t_{\text{obs}}, t_{\text{act}}, t_{\text{wait}}$ observe, act, wait · T_{saving} time saved by running steps at the same time · $R^{\text{hit}}, R^{\text{miss}}, R^{\text{store}}$ prompt tokens from the cache, not in it, stored into it · W written tokens · p price per token of each class · τp_{env} machine hours × hourly price · $\text{Pr}[\text{success}]$ success rate

SAME SOURCE — IN BOTH EQUATIONS

fix it, and time and money fall together

- Passes N , calls per pass c_k , reading, writing, cache hits, failures — every one of them lives in the token counts

(Abhyankar, Qi & Zhang, 2026; Anthropic, 2026b)

SLOW BUT NOT EXPENSIVE — NO TOKENS, ONLY MACHINE HOURS

fix it, and only time falls

- Queueing, page loads, fixed sleeps, tool tails — none produces a token; the machine behind them is billed by the hour, usually under 5% of the API bill

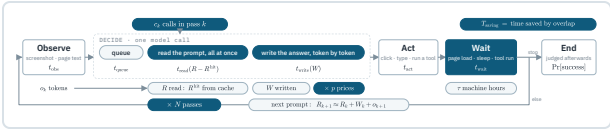
MONEY FOR TIME OR FOR ACCURACY — OPPOSITE SIGNS

paying for speed shortens only the writing segment; paying for accuracy costs more tokens per point as accuracy rises

- Fast mode: up to 2.5× faster writing (vendor-stated) at 2× the price, reading unchanged

(Anthropic, 2026b; OpenAI, 2026; accuracy: Appendix A5)

The levers, and what is still unmeasured



from the three relationships → here: what fixing each lever buys, and the holes in the evidence → to: Part 2, what the literature has done with each lever

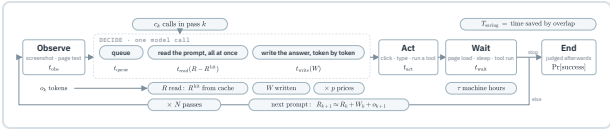
Seven levers follow from the equations; each saves a known term and carries a known risk. Part 2 asks how far the literature has pushed each one. On the right: what no measurement has settled — gaps in the evidence, not claims of a research gap.

LEVER	TIME	MONEY	THE CATCH
Fewer passes — several actions per call, skip navigation-only steps	yes	yes, the quadratic term	which steps need no thinking?
Read less — trim history, shrink screenshots, cache the prefix	yes	yes	forgets; trimming invalidates the cache
Write less — less thinking, smaller model	yes	yes	accuracy may fall
Fewer calls per pass — drop judging and reflection	yes	yes	fewer errors caught
Overlap the waiting — pre-load, parallel tools, page-ready events	yes	no	only where the environment dominates
Buy a fast or priority tier	writing only	no — 2× more	reading unchanged; cache dropped
Stop retrying failures — early stop, detect dead loops	yes	yes	stops some attempts that would have succeeded

NOT YET MEASURED

- **Read / write time split and cache hit rate for a frontier API model** — the only split is on local 27–31B models (Yuan et al., 2026); screenshot-agent measurements do not report cache state.
- **How much slower than a person** — no benchmark times agent and human on the same tasks; the closest, AXIS’s small user study, has a UI agent 1.69× slower than manual work on easy tasks (Lu et al., 2025).
- **Whether the benchmark’s fixed sleep sits inside “action” time** in OSWorld-Human — if so, “environment under 3.5%” understates the waiting.
- **How much slowness costs accuracy** — OSWorld 2.0 records the stale-screen failure mode without its share (XLANG Lab, 2026); the rebound from a smaller model has no direct evidence.

References · Part 1 (1 of 2)



author-year tags on the pages refer to these entries · published version where one exists, otherwise the arXiv number and the authors' institutions

1. Abhyankar, R., Qi, Q., & Zhang, Y. (2026). OSWorld-Human: Benchmarking the efficiency of computer-use agents. *Proceedings of the 9th Conference on Machine Learning and Systems (MLSys 2026)*. arXiv:2506.16042. University of California, San Diego.

2. Anthropic. (2026a, May 13). *Best practices for computer and browser use with Claude*. <https://claude.com/blog/best-practices-for-computer-and-browser-use-with-claude>

3. Anthropic. (2026b). *Claude API pricing; Fast mode* (developer documentation, read 28 September 2026). <https://platform.claude.com/docs/en/about-claude/pricing>; <https://platform.claude.com/docs/en/build-with-claude/fast-mode>

4. Bian, S., Yan, M., Jayarajan, A., Pekhimenko, G., & Venkataraman, S. (2025). What limits agentic systems efficiency? arXiv:2510.16276. University of Wisconsin–Madison; University of Toronto; NVIDIA.

5. Chang, C., Zhou, Y., Fu, K., An, D., Feng, T., Lu, H., Yao, S., Guo, P., Yu, Y., Shan, Y., Li, B., Yuan, B., & Wang, W. (2026). From LLM inference to agentic workloads: Characterization and implications for serving systems (AgentSysBench). arXiv:2608.15127. Hong Kong University of Science and Technology; Alibaba Group; ByteDance.

6. Chen, L., Zaharia, M., & Zou, J. (2024). FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*. arXiv:2305.05176. Stanford University.

7. DeepSeek. (2026). *Models & pricing* (API documentation, read 28 September 2026); *DeepSeek-V4.1-Flash release* (10 September 2026). https://api-docs.deepseek.com/quick_start/pricing; <https://api-docs.deepseek.com/news/news260910>

8. Erol, M. H., El, B., Suzgun, M., Yuksekgonul, M., & Zou, J. (2026). Cost-of-Pass: An economic framework for evaluating language models. *The Fourteenth International Conference on Learning Representations (ICLR 2026)*. arXiv:2504.13359. Stanford University.

9. Feng, G., Mao, H., Dutta, P., & Gonzalez, J. E. (2026). Concurrency without model changes: Future-based asynchronous function calling for LLMs (AsyncFC). arXiv:2605.15077. University of California, Berkeley.

10. Google. (2026). *Gemini Developer API pricing* (read 28 September 2026). <https://ai.google.dev/gemini-api/docs/pricing>

11. Kang, H., Li, Z., Yang, X., Xu, W., Chen, Y., Wang, J., Chen, B., Krishna, T., Xu, C., & Arora, S. (2026). ThunderAgent: A fast, simple, and program-aware agentic inference system. *Proceedings of the 43rd International Conference on Machine Learning (ICML 2026)*, poster. arXiv:2602.13692. Georgia Institute of Technology; University of Illinois Urbana-Champaign; Carnegie Mellon University; Together AI.

12. Kapoor, S., Stroebl, B., Kirgis, P., Nadgir, N., Siegel, Z. S., Wei, B., ... Narayanan, A. (2026). Holistic Agent Leaderboard: The missing infrastructure for AI agent evaluation. *The Fourteenth International Conference on Learning Representations (ICLR 2026)*. arXiv:2510.11977. Princeton University et al.

13. Kim, S., Moon, S., Tabrizi, R., Lee, N., Mahoney, M. W., Keutzer, K., & Gholami, A. (2024). An LLM compiler for parallel function calling. *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*. arXiv:2312.04511. University of California, Berkeley.

14. Le Sellier de Chezelles, T., Gasse, M., Lacoste, A., Caccia, M., Drouin, A., Boisvert, L., ... Chapados, N. (2025). The BrowserGym ecosystem for web agent research. *Transactions on Machine Learning Research*. arXiv:2412.05467. ServiceNow Research et al. (figure not re-verified; used only in Appendix A1)

15. Lee, N., Erdogan, L. E., John, C. J., Krishnapillai, S., Mahoney, M. W., Keutzer, K., & Gholami, A. (2026). Agentic test-time scaling for WebAgents. arXiv:2602.12276. University of California, Berkeley.

16. Liu, B., Qiu, H., Goiri, Í., Fonseca, R., Bianchini, R., & Choukse, E. (2026). Agentic coding in the wild: Characterizing GitHub Copilot traces at production scale. arXiv:2608.00101. University of Illinois Urbana-Champaign; Microsoft Azure Research.

References · Part 1 (2 of 2)

continued

17. Lu, J., Zhang, Z., Yang, F., Zhang, J., Wang, L., Du, C., Lin, Q., Rajmohan, S., Zhang, D., & Zhang, Q. (2025). AXIS: Efficient human-agent-computer interaction with API-first LLM-based agents. *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 7711–7743. <https://doi.org/10.18653/v1/2025.acl-long.381>. Microsoft.

18. Luo, M., Shi, X., Cai, C., Zhang, T., Wong, J., Wang, Y., Wang, C., Huang, Y., Chen, Z., Gonzalez, J. E., & Stoica, I. (2026). Agentix: An efficient serving engine for LLM agents as general programs. *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. University of California, Berkeley; Google DeepMind; Shanghai Jiao Tong University.

19. Meta. (2026, September 8). *Introducing Muse, your personal AI agent* [Newsroom post]. <https://about.fb.com/news/2026/09/introducing-muse-personal-ai-agent/>

20. OpenAI. (2025). *Introducing Operator* (23 January 2025); *Introducing ChatGPT agent* (17 July 2025). <https://openai.com/index/introducing-operator/>; <https://openai.com/index/introducing-chatgpt-agent/>

21. OpenAI. (2026). *API pricing*; *API changelog* (fast mode, 30 July 2026; Ultrafast, 13 August 2026) (read 28 September 2026). <https://developers.openai.com/api/docs/pricing>; <https://developers.openai.com/api/docs/changelog>

22. Vendor price pages used for Appendix A3 (all read 27 September 2026): AWS, *Amazon EC2 T3 instances*; Browser Use, *Pricing* and *API v4: create browser session*; Browserbase, *Pricing* and *Billing plans*; Daytona, *Pricing* and *Billing*; E2B, *Pricing*; Modal, *Pricing* and *Sandbox resources*.

23. Winston, C., Wang, R. Y., Mirhoseini, A., & Kozyrakis, C. (2026). Agent JIT compilation for latency-optimizing web agent planning and scheduling. *Proceedings of the 43rd International Conference on Machine Learning (ICML 2026)*, PMLR 306. arXiv:2605.21470. Stanford University.

24. Wong, M., Hsieh, K., Nath, S., & Netravali, R. (2026). Skim: Speculative execution for fast and efficient web agents. arXiv:2605.16565. Princeton University; Microsoft Research.

25. xAI. (2026). *Grok 4.7* model page, SpaceXAI Docs (read 28 September 2026). <https://docs.x.ai/developers/models/grok-4.7>

26. XLANG Lab. (2026). OSWorld 2.0: Benchmarking computer use agents on long-horizon real-world tasks. arXiv:2606.29537. The University of Hong Kong (authored as “XLANG Lab and Collaborators”; the full author list is in the paper’s Appendix A).

27. xlang-ai. (2026). *OSWorld* [Source code], `desktop_env/desktop_env.py` and `run.py` at commit b138d348. <https://github.com/clang-ai/OSWorld>

28. Xu, B., Xue, Z., Chen, D., Fu, C., Wu, C., Huang, C., ... Zhang, N. (2026). TokenPilot: Cache-efficient context management for LLM agents. arXiv:2606.17016. Zhejiang University; University of Electronic Science and Technology of China; Xidian University; HomologyAI.

29. Yuan, Y., Nayak, A., Kundu, S., & Talati, N. (2026). Agentic AI workload characteristics. arXiv:2605.26297. University of Illinois Urbana-Champaign; Gimlet Labs; Intel.

30. Zhang, C., Huang, H., Ni, C., Mu, J., Qin, S., He, S., ... Zhang, D. (2026). UFO2: The desktop AgentOS. *Transactions on Machine Learning Research*. arXiv:2504.14603. Microsoft.

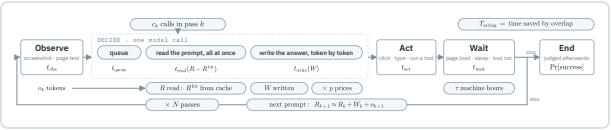
31. Zhu, K., Jacob, M., Ma, C., Pan, Y., Wang, S., Krishnamurthy, A., & Kasikci, B. (2026). TraceLab: Characterizing coding agent workloads for LLM serving. arXiv:2606.30560. University of Washington.

```
graph LR
    Observe[Observe] -- "o_t tokens" --> Queue[queue]
    Queue -- "read the prompt, all at once" --> Write[write the answer, token by token]
    Write -- "t_{act}(W)" --> Wait[Wait]
    Wait -- "t_{wait}(W)" --> End[End]
    Wait -- "t_{wait}(W)" --> Observe
    Wait -- "t_{wait}(W)" --> End
    Wait -- "t_{wait}(W)" --> End
    Wait -- "t_{wait}(W)" --> End
```



← back

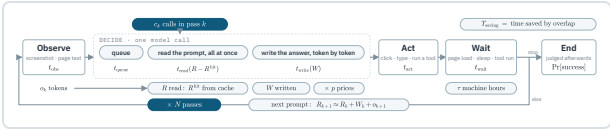
A0 · Page 2’s variables, the step each belongs to, and the pages that expand it



every symbol on page 2 · the same terms light up in the loop thumbnail at the top right of each page

SYMBOL	MEANING	STEP IN THE LOOP	EXPANDED ON
N	passes per task	the loop	pages 3–4, 6, 10 · A1-1
k, j	index of the pass; index of the call inside the pass	the loop; decide	—
c_k	model calls in pass k (planning, judging, reflecting, sampling), set by the harness	decide	pages 3–4, 7 · A1-1, A3-2
ℓ_{kj}	elapsed time of call j in pass k	decide	page 5 · A1-2
t_{queue}	waiting at the provider before processing; unbilled; set by the provider’s load	decide · queue	pages 5, 9 · A1-2
t_{read}	reading the prompt, all tokens at once (prefill); only tokens not served from the cache	decide · read	pages 5–6 · A1-2, A2
t_{write}	writing the answer one token at a time (decode), thinking included	decide · write	pages 5–6 · A1-2
$t_{\text{obs}}, t_{\text{act}}, t_{\text{wait}}$	producing the observation; executing the action; page loads, fixed sleeps, tool runs	observe; act; wait	pages 5–6, 9 · A1-3
T_{saving}	time saved because steps run at the same time; zero when everything runs in sequence	the loop	pages 3, 5, 10 · A1-3
$R = R^{\text{hit}} + R^{\text{miss}} + R^{\text{store}}$	tokens a call reads: from the cache (hit), not in it (miss), stored into it (store)	decide · read	pages 7–8 · A3-1, A3-2
$o_k, \bar{o}$	tokens the observation of pass k adds to the prompt; their mean per pass	observe	pages 5, 7 · A1-2, A3-1
$W, \bar{w}$	tokens written by a call, thinking included; mean per pass	decide · write	pages 5, 7 · A3-1
$p_{\text{hit}}, p_{\text{miss}}, p_{\text{store}}, p_{\text{write}}$	price per token of each class, for the model and tier used	decide	pages 7–8 · A3-2, A3-3
τ, p_{env}	hours the environment machine is billed (at least T_{task}); its hourly price	wait	page 8 · A3-3
$\text{Pr}[\text{success}], s_0$	probability that one attempt completes the task; its level before a change	end (judged afterwards)	pages 7–8, 10 · A3-2

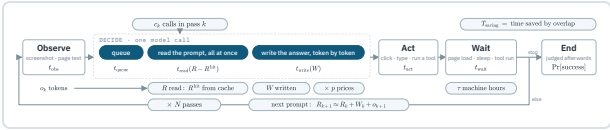
A1 · Slowness, every cause — I. steps and II. calls per step



the complete table behind pages 3–4 · three kinds of agent: screenshot desktop agent, text web agent, coding agent with a prompt cache

ID	CAUSE	MECHANISM	EVIDENCE, WITH THE CONDITIONS OF THE MEASUREMENT	SOURCE	DOMINANT IN
I-1	The task is long	Hour-scale tasks take hundreds of passes	OSWorld 2.0, 108 long tasks: Claude Opus 4.7 with maximum thinking, one action per step, 500-step cap: 318 tool calls per task on average, i.e. 318 steps; with several actions per call, Claude Opus 4.8 takes 103 steps (481.8 calls) and GPT-5.5 95.2 steps (149.8 calls)	(XLANG Lab, 2026)	screenshot, text
I-2	One action per pass	Every click is a whole pass: one observation, one model call, one wait	OSWorld 2.0: with one action per step Claude Opus 4.7 needs 318 steps; with batched actions the same model needs 160.7 and Claude Opus 4.8 needs 103	(XLANG Lab, 2026)	screenshot, text
I-3	Navigation-only passes still call the model	Paging, scrolling and opening the next page need no thinking but cost a call each	Skim’s profile of three text web agents (Browser-Use, AgentOccam, WebVoyager; GPT-4o) on 151 WebVoyager live-site tasks: 66.7% of the steps in the median task are pure navigation	(Wong et al., 2026)	text
I-4	Idling and dead loops	Element not found, repeated retries; steps rise without progress	OSWorld-Human’s records of the GTA1 harness (o3 plans and judges, GTA1-7B locates elements) on 39 OSWorld desktop tasks: in failed tasks that exceeded 50 steps, 66% of steps were wasted repeats; one element-locating loop repeated the same step 18 times — 27 minutes and \$8.47 at list price without caching	(Abhyankar, Qi & Zhang, 2026)	screenshot
I-5	Acting on a stale screen	The screenshot is taken before the page changes; the agent clicks on the old layout and must repair	OSWorld 2.0 records “stale interface state” as a failure mode; no share reported	(XLANG Lab, 2026)	screenshot
II-1	Multi-call harness	One step = plan + judge + reflect, several calls in series	OSWorld-Human, two harnesses: GTA1 runs 4 parallel planners per step, retries up to 3 rounds, then one judging call picks — 4–12 planning calls per judging call; Agent S2 adds a reflection call per step, so planning is 53% and reflection 34% of task time	(Abhyankar, Qi & Zhang, 2026)	screenshot
II-2	Sampling for accuracy	5–20 candidates per step, then pick one	Same model and harness (gpt-oss-120b, ReAct) on 165 WebArena-Lite tasks: 1 → 10 candidates per step raises success from 38.8% to 43.2% and tokens per task from 96K to 920K — 9.6× the tokens for 4.4 points	(Lee et al., 2026)	text
II-3	Coding agents call repeatedly within a turn	The model is called back and forth inside one user turn	GitHub Copilot coding agent production telemetry (first week of June 2026, 13.5 million sessions): 6.6 model calls per user turn on average	(Liu et al., 2026)	coding

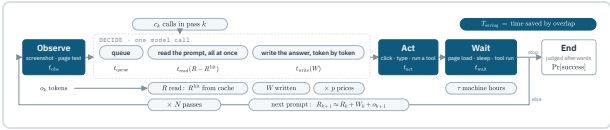
A1 · Slowness, every cause — III. each call is slow



the complete table behind page 5

ID	CAUSE	MECHANISM	EVIDENCE, WITH THE CONDITIONS OF THE MEASUREMENT	SOURCE	DOMINANT IN
III-1	Queueing	The request waits at the provider; unbilled, but counted as model time	Client-side measurement of 15 models across 5 providers: requests of the same length differ in latency by up to 69× depending on when they are sent	(Bian et al., 2025)	all
III-2	Reading (the prompt is re-processed every call)	Every call re-reads the whole history; more screenshots, slower	OSWorld-Human, multi-call harnesses: later steps up to 3× slower because the prompt at step k holds the $k - 1$ earlier screenshots — “dominated by prefill”. One screenshot is 1,000–1,800 tokens. AgentSysBench’s WebArena agent (Kimi-K2.6): observation switched from a single format to accessibility tree + HTML + screenshot → input 4.8×, model share of time 46.9% → 61.6%. Under load the cache is evicted and re-read: SWE-Agent / OpenHands running GLM-4.6 on 8 H100s for SWE-bench Lite, request latency up to 7.14× as concurrency rises	(Abhyankar, Qi & Zhang, 2026; Anthropic, 2026a; Chang et al., 2026; Kang et al., 2026)	screenshot; text with rich observations; coding under load
III-3	Writing (one token at a time)	Token-by-token generation, strictly serial	Locally served ReAct agents (Qwen3.6-27B / Gemma4-31B, vLLM, 2 H100s) on five benchmarks: with the cache warm, generation is 91–98.6% of model time. Windows desktop agent UFO2 (GPT-4o / o1 API): about 10 s per model call, the largest item per step in every configuration	(Yuan et al., 2026; Zhang et al., 2026)	coding; single-call screenshot
III-4	Thinking modes and larger models	Thinking writes more tokens; larger models are slower per token	OSWorld 2.0, same 108 tasks: Claude Opus 4.8 writes 224K output tokens per task, GPT-5.5 37K. Holistic Agent Leaderboard, 9 benchmarks, 21,730 runs: raising reasoning effort lowered accuracy in 21 of 36 pairs	(XLANG Lab, 2026; Kapoor et al., 2026)	all

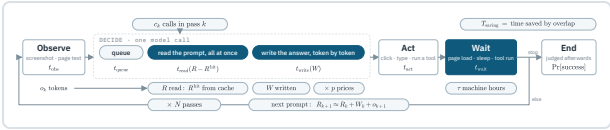
A1 · Slowness, every cause — IV. environment, V. serial



the complete table behind page 5 (environment and serial structure)

ID	CAUSE	MECHANISM	EVIDENCE, WITH THE CONDITIONS OF THE MEASUREMENT	SOURCE	DOMINANT IN
IV-1	Producing the observation	Extracting the accessibility tree or detecting elements takes time	OSWorld desktop applications: 3–26 s to generate one accessibility tree; the screenshot itself is under 2% of task time. UFO2: OmniParser element detection adds about 1 s per step	(Abhyankar, Qi & Zhang, 2026; Zhang et al., 2026)	screenshot
IV-2	Browser execution and page load	After a click the agent waits for the page	151 WebVoyager live-site tasks: per step, median browser action 6.6 s vs model call 4.7 s. WebArena with GenericAgent + Claude 3.5 Sonnet: 7.6 s of a 12.2 s step in the browser (this figure was not re-verified)	(Wong et al., 2026; Le Sellier de Chezelles et al., 2025)	text
IV-3	Fixed sleeps	The agent cannot tell when the page is ready, so it sleeps a fixed number of seconds	OSWorld’s environment code sleeps 2 s after every action; OSWorld 2.0 prescribes 3 s, so 318 steps are about 16 minutes of pure waiting (calc.). AgentSysBench: a single-call GUI agent (ReAct, Kimi-K2.6 API) spends over 70% of its OSWorld execution time in the desktop sandbox	(xlang-ai, 2026; XLANG Lab, 2026; Chang et al., 2026)	screenshot (caused by the benchmark harness)
IV-4	Tool tails	Tests and builds take minutes	TraceLab, 4,265 Claude Code / Codex sessions from 43 developers (September 2025 – June 2026): tool calls over 1 minute are 4% of calls but 85% of tool time; a request averages 4.3 minutes — tools 2.5 (59.8%), model 1.7 (41.0%)	(Zhu et al., 2026)	coding
V	Serial structure	The four stages never overlap, so task time is the sum of every term, not the largest one; fixing one term saves only its share	Concurrency within a turn is 1.15 in Copilot telemetry. Separately, 80–92% of a coding session’s elapsed time is the human thinking between turns — strip it before reading any total	(Liu et al., 2026; Zhu et al., 2026)	all

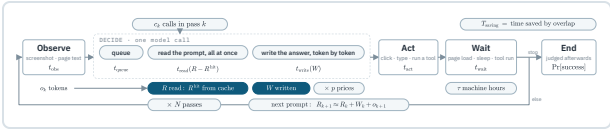
A2 · Where the arrows land, with every measurement condition



the complete table behind page 6

	SCREENSHOT DESKTOP AGENT, MULTI-CALL HARNESS	TEXT WEB AGENT	CODING AGENT WITH A PROMPT CACHE
heaviest term	Reading × calls per step × steps. OSWorld-Human timed two multi-call harnesses (GTA1, Agent S2) step by step on 39 OSWorld desktop tasks: 87–97% of task time in planning, judging and reflection calls; screenshots and actions together under 3.5% (Abhyankar, Qi & Zhang, 2026)	Browser execution plus waiting. Skim measured three text web agents (GPT-4o) on 151 WebVoyager live-site tasks: per step, median browser action 6.6 s and model call 4.7 s (Wong et al., 2026)	Writing, or tool tails, depending on the tools. With the context cache on, token-by-token generation is 91–98.6% of model time (locally served models) (Yuan et al., 2026); in Claude Code / Codex records a request spends 59.8% in tools and 41.0% in the model (Zhu et al., 2026); in GitHub Copilot, whose tools are light, agent time is 13.7% model and 2% tools (Liu et al., 2026)
second	Writing: with thinking on, more reasoning tokens per step, and generation is one token at a time	Reading, once the observation grows: AgentSysBench switched a WebArena agent from one observation format to accessibility tree + HTML + screenshot — input 4.8×, model share 46.9% → 61.6% (Chang et al., 2026); the Browser-Use harness spends 73% of its latency in model calls (Winston et al., 2026)	Reading, when load evicts the cache and the whole context is re-read: ThunderAgent measured request latency up to 7.14× under cache thrash (Kang et al., 2026)
negligible	Producing the observation and executing the action: under 3.5% together (Abhyankar, Qi & Zhang, 2026)	Nothing: model and browser are close; which is larger depends on observation size and model speed	Navigation-only steps: a coding agent has no pages to turn
counter-example	A single-call agent is different: AgentSysBench’s one-call GUI agent (Kimi-K2.6) spends over 70% of its OSWorld time in the desktop sandbox (Chang et al., 2026), because the harness sleeps after every action — 2 s in OSWorld’s code (xlang-ai, 2026), 3 s in OSWorld 2.0 (XLANG Lab, 2026)	Browser-Use: 73% of latency in model calls, not the browser (Winston et al., 2026)	Two production records point opposite ways: heavy tools in Claude Code / Codex (tools 59.8% > model 41.0%) (Zhu et al., 2026); light tools in Copilot (model 13.7% > tools 2%) (Liu et al., 2026). The difference is the kind of tool (tests and builds vs file reads), not a contradiction
in one line	Slow in reading: every step re-reads the screenshot history, more as the task goes on	Slow in the environment (browser and page waits); once the observation grows, slow in reading again	Reading is cached away; slow in writing and in the tools

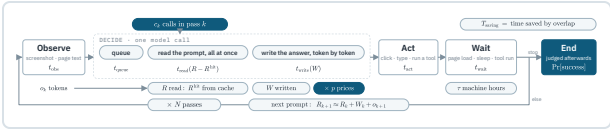
A3 · Cost, every cause — 1. tokens read and 2. tokens written



the complete table behind page 7 · prices are list prices from the vendors’ own pages, September 2026

ID	CAUSE	MECHANISM	EVIDENCE, WITH THE CONDITIONS OF THE MEASUREMENT	SOURCE
1-1	History re-sent	Call k reads everything from the $k - 1$ earlier steps; total reading $\approx N^2$	“Cost grows quadratically with the number of steps” — OSWorld-Human, real runs	(Abhyankar, Qi & Zhang, 2026)
1-2	Large screenshots	1,000–1,800 tokens per image; 100 images fill a 200K context	Anthropic’s engineering guidance for computer and browser use	(Anthropic, 2026a)
1-3	Rich observation formats	Accessibility tree + HTML + screenshot together: 4.8× the input	AgentSysBench, WebArena agent, Kimi-K2.6	(Chang et al., 2026)
1-4	Sampling	Several candidates per step, mostly read tokens	gpt-oss-120b + ReAct on 165 WebArena-Lite tasks: 1 → 10 candidates per step, 96K → 920K tokens per task	(Lee et al., 2026)
1-5	The cache is not a cure	However high the hit rate, a long history keeps reading dominant; a miss re-reads everything	TraceLab, 4,265 Claude Code / Codex sessions: prefix-cache hit rate 95.7%, prefix tokens still 59.5% of list-price cost; on a miss the re-read is 3.8× the genuinely new content	(Zhu et al., 2026)
2-1	Thinking modes	Thinking tokens are billed as output; when they cannot be switched off they are a fixed overhead	OSWorld 2.0, same tasks: Claude Opus 4.8 224K output tokens per task, GPT-5.5 37K; Claude Opus 5.5: “thinking cannot be disabled and is billed as output”	(XLANG Lab, 2026; Anthropic, 2026b)
2-2	But writing is not always the larger bill	In an uncached screenshot agent, output was 31% of the total	OSWorld-Human, GTA1: the paper’s \$2.43 counts output only; Table 3’s input and output give \$7.87	(Abhyankar, Qi & Zhang, 2026)

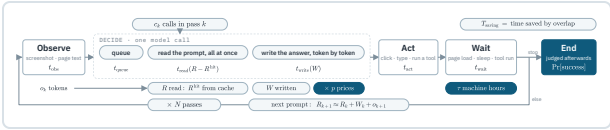
A3 · Cost, every cause — 3. calls, 4. failures, 5. unit price



the complete table behind page 7 (calls, failures, unit price)

ID	CAUSE	MECHANISM	EVIDENCE, WITH THE CONDITIONS OF THE MEASUREMENT	SOURCE
3	Number of calls	$N \cdot c_k$ scales both token bills	GTA1: 4 parallel planners per step, up to 3 retry rounds, one judging call → 4–12 planning calls per judging call, i.e. 5–13× the calls of a single-call harness (calc.). Agentic test-time scaling: 5–20 candidates per step multiply the calls by the candidate count. The same extra calls cost time and money at once	(Abhyankar, Qi & Zhang, 2026; Lee et al., 2026)
4-1	Failed attempts are billed	Cost per success = cost per attempt ÷ success rate	OSWorld 2.0: the best agent (Claude Opus 4.8) costs about \$72.4 per attempt at 20.6% completion → about \$351 per success (calc.). An accounting conversion, not the real price of retrying one task until it succeeds	(XLANG Lab, 2026)
4-2	Idle steps are billed in full	Every pass of a dead loop is a full read and write	OSWorld-Human, GTA1 on 39 OSWorld tasks: one element-locating loop repeated the same step 18 times — 27 minutes, \$8.47 at list price without caching; in failed tasks over 50 steps, 66% of steps were such repeats	(Abhyankar, Qi & Zhang, 2026)
4-3	Evaluation itself is too expensive to repeat	Single runs, no repeats	Holistic Agent Leaderboard: 9 benchmarks, 21,730 runs, one run per configuration, about \$40,000 in total; Claude Opus 4.1 not run on Online-Mind2Web because the estimate was \$20,000	(Kapoor et al., 2026)
5-1	Model tier	Claude Opus 5.5 \$4 / \$20 vs GPT-6 Luna \$0.10 / \$0.50 per million tokens read / written	Vendor price pages, 27 September 2026	(Anthropic, 2026b; OpenAI, 2026)
5-2	Fast mode	2× on both vendors; switching speed tiers invalidates the cache	Anthropic: Claude Opus 5.5 \$8 / \$40, up to 2.5× output speed (vendor-stated), first-token wait unchanged. OpenAI: 2× on all listed models; “up to 2.5×” stated only for GPT-5.6 Sol	(Anthropic, 2026b; OpenAI, 2026)
5-3	Cache write and read	Latest model of each vendor, per million tokens, normal read → cached read (ratios calc.): DeepSeek-V4.1-Flash \$0.30 → \$0.006 (0.02×; peak rate, both halve off-peak) Claude Opus 5.5 \$4 → \$0.20 (0.05×; other Claude models 0.1×, Fable 5.1 0.025×; cache writes 1.25× for 5 min, 2× for 1 hour) GPT-6 Sol \$2 → \$0.20 and GPT-6 Luna \$0.10 → \$0.01 (0.1×; cache writes 1.25×) Gemini 3.8 Flash \$0.75 → \$0.075 (0.1×; plus \$0.50 per million tokens per hour of storage; introductory to 31 Dec 2026) Grok 4.7 \$2 → \$0.50 (0.25×)	Vendor price pages, read 28 September 2026	(DeepSeek, 2026; Anthropic, 2026b; OpenAI, 2026; Google, 2026; xAI, 2026)
5-4	Long-context surcharge	OpenAI: 2× the read price above 272K tokens	Vendor price page	(OpenAI, 2026)

A3 · Cost — 6. the environment machine, and the four conventions

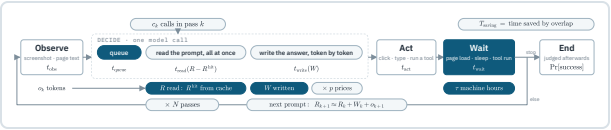


the complete tables behind page 8 · vendor price pages read 2026-09-27 · per-task amounts and shares are our own arithmetic (calc.)

ENVIRONMENT	PRICE (VENDOR PAGE)	1-HOUR TASK	15-MIN TASK	SHARE OF A \$7.87 API BILL	SHARE OF \$72.4
Browser Use cloud browser	\$0.02 per browser-hour, billed by the minute, 1-minute minimum; traffic extra (residential proxy \$5/GB, direct \$0.20/GB); sessions capped at 240 minutes	\$0.020	\$0.005	0.25%	0.03%
AWS t3.medium (2 vCPU / 4 GiB, Linux, us-east-1)	\$0.0418 per hour on the T3 product page; another AWS page says \$0.0416 (0.5% apart)	\$0.042	\$0.010	0.5%	0.06%
Browserbase cloud browser	\$20 per month for 100 hours, then \$0.12 per hour; \$99 per month for 500 hours, then \$0.10; billed by the minute	\$0.12	\$0.03	1.5%	0.17%
E2B / Daytona sandbox (2 vCPU / 4 GiB)	E2B \$0.000014 per vCPU-second + \$0.0000045 per GiB-second; Daytona \$0.0504 per vCPU-hour + \$0.0162 per GiB-hour — both \$0.1656 per hour	\$0.166	\$0.041	2.1%	0.23%
Modal sandbox (1 physical core = 2 vCPU, 4 GiB)	\$0.00003942 per core-second + \$0.00000667 per GiB-second → \$0.238 per hour; billed on the larger of requested and used	\$0.238	\$0.060	3.0%	0.33%
AWS t3.2xlarge (8 vCPU / 32 GiB, OSWorld 2.0's default instance)	\$0.3341 per hour	\$0.334	\$0.084	4.2%	0.46%
OpenAI hosted container (Hosted Shell / Code Interpreter, 4 GB)	\$0.12 per 20-minute session → \$0.36 per hour; 1 GB \$0.03, 16 GB \$0.48, 64 GB \$1.92 per 20 minutes	\$0.36	\$0.12	4.6%	0.50%

The four accounting conventions (page 8), in one line each: output only vs all tokens — 3.2× (OSWorld-Human, \$2.43 vs \$7.87); per attempt vs per success — 4.9× (OSWorld 2.0, \$72.4 ÷ 20.6%); uncached vs cached — 4–50× on the read price by vendor (calc.), far less on the bill (TraceLab: 95.7% hits, prefix still 59.5% of cost); standard vs fast mode — 2× (both vendors), buying only writing speed.

A5 · Each cause’s effect on time and on money



the complete table behind pages 9–10

CAUSE	EFFECT ON TIME	EFFECT ON MONEY	RELATIONSHIP
Many passes (N)	Roughly linear: one more step is one more call and one more wait (later steps somewhat slower, up to 3×)	Write tokens linear in N ; read tokens quadratic, because every step re-reads the whole history	same source — double the steps: about 2× the time, about 4× the read bill (uncached)
Many calls per pass (c_k)	linear	linear	same source
Reading a lot (history, screenshots, rich observations)	Reading time, longer every step	Read-token bill	same source — the same tokens cost time and money
Writing a lot (thinking)	Generation time, roughly 12 ms per token	Write-token bill at 5× the read price	same source , and the dearest time: writing is the slowest and the most expensive token
Cache hits	Saves reading time	Read price 0.02–0.25× of a normal read, by vendor (A3)	same source , but fragile: editing the history or switching to fast mode invalidates it
Queueing	slow	free	slow but not expensive ; escaping it means a fast or priority tier at 2×
Environment waits (page loads, fixed sleeps)	slow	No API cost; machine time by the hour — CPU sandboxes \$0.02–0.36 per hour, usually under 5% of the API bill	slow but not expensive
Tool tails (tests, builds)	slow	No API cost	slow but not expensive
Failures, idling, retries	slow	expensive	same source , and it turns “per attempt” into “per success”
Fast mode	Writing up to 2.5× faster (vendor-stated); reading unchanged	2× the price	money for time
Bigger models, more thinking	slower	dearer	money and time for accuracy , convex: marginal tokens per point 101K → 575K on one model; 6× / 9× / 9.6× tokens for a few points; more thinking not always more accurate (Lee et al., 2026; XLANG Lab, 2026; Kapoor et al., 2026)
Smaller models	faster	cheaper	but if accuracy falls, steps and retries rise — possibly slower and dearer again (inference, no direct evidence)



← back